GLB 3rd Year Assessment 2 - Slot 2

Contest Problem Report

Contest Code	GLB3A2S2
Total Problems	5
Coding Problems	5

Problem Name: Is the i-th bit set

Problem Statement:

You are given two integers, *n* and *i*. Your task is to determine whether the ith **bit** (0-indexed from the least significant bit) in the **binary form of n** is *1*. Return *true* if the bit is set, otherwise return *false*.

Function Declaration

Function Name

__TEXT_PLACEHOLDER0 – This function checks whether the ith bit (0-indexed from the least significant bit) in the binary representation of an integer *n* is set.

Parameters

- *n*: An integer whose binary representation is to be examined.
- *i*: An integer indicating the bit position (0-indexed from the least significant bit).

Return Value

- Returns *true* if the ith bit of *n* is set (i.e., equals 1).
- Returns *false* otherwise.

`The input and output formats given below are only if you want to test using custom inputs.`

Constraints

- $1 \leq T \leq 10^5$
- $1 \leq n \leq 3*10^9$
- $0 \leq i \leq 31$

Input Format

- The first line contains a single integer *T* — the number of test cases.
- Each test case consists of two lines containing integers:
    - *n* — the number
    - *i* — the bit position to check

Output Format

- For each test case, print *true* if the ith bit of *n* is set.
- Otherwise, print *false*.

Sample Input 1:

1 12 2

Sample Output 1:

true

Explanation:

Binary representation of 12 is **1100**. The bit at index 2 (from the right) is **1**, so it is set.

Sample Input 2:

1 7 3

Sample Output 2:

false

Explanation:

Binary representation of 7 is **0111**. The bit at index 3 is **0**, so it is not set.

Sample Input 3:

1 20 4

Sample Output 3:

true

Explanation:

Binary representation of 20 is **10100**. The bit at index 4 is **1**, so it is set.

Problem Name: Longest Substring Without Repeating Characters

Problem Statement:

Given a string S of length N, you have to output the length of the **longest substring** of S with **non-repeating characters**.

A **substring** is a contiguous sequence of characters within the string. The substring must not contain any character that appears more than once.

Example:

`S = "abcabcbb"`

- Longest substring without repeating characters: `"abc"`
- Output: 3

Function Declaration

Function Name

longestUniqueSubstring – This function computes the length of the longest contiguous substring of a given string that contains no repeating characters.

Parameters

- S : A string of length N consisting of characters (typically lowercase English letters, but the logic is character-agnostic).

Return Value

- Returns an integer representing the **maximum length** of a substring of S such that **each character appears at most once** in that substring.

Constraints

- $2 \leq N \leq 10^5$
- S contains valid characters (no restriction on character set unless specified)
- The solution should be efficient enough to handle large input sizes

Input Format

- The first and only line of input contains a single string S.

Output Format

- Output the length of the longest substring with no repeating characters.

Sample Input 1:

abcabcbb

Sample Output 1:

3

Explanation:

The answer is "abc" with length 3.

Sample Input 2:

bbbb

Sample Output 2:

1

Explanation:

Sample Input 3:

abcccdefg

Sample Output 3:

5

Explanation:

Problem Name: Bucket and Water Flow

Problem Statement:

Alice has a bucket of water initially having W litres of water in it. The maximum capacity of the bucket is X liters.

Alice turned on the tap and the water starts flowing into the bucket at a rate of Y litres/hour. She left the tap running for exactly Z hours. Determine whether the bucket has been overflown, filled exactly, or is still left unfilled.

Input Format

- The first line of input will contain a single integer T, denoting the number of test cases. The description of the test cases follows.
- Each test case consists of a single line of input containing four space-separated integers $W, \ X, \ Y, \ Z$.

Output Format

For each test case, print the answer on a new line:

- If the bucket has overflown print `overflow`
- If it is exactly filled print `filled`
- If it is still unfilled, print `unfilled`

You may print each character of the string in uppercase or lowercase (for example, the strings `filled`, `FIlled`, `fiLLed`, and `FILLED` will all be treated as identical).

Constraints

- $1 \leq T \leq 1000$
- $1 \leq W,X,Y,Z \leq 1000$

Subtasks

- **Subtask 1 (100 points):**
  - Original constraints.

Sample Input 1:

4 1 2 3 4 10 70 10 6 2 100 4 3 3 3 2 1

Sample Output 1:

overFlow filled Unfilled overflow

Explanation:

Test case 1: Initially the bucket had 1 litre of water, we then added 3 litres of water for 4 hours. Thus, the total bucket inflow was $1 + 3 \times 4 = 13$ litres. Since this is greater than the capacity of 2 litres, the

bucket will `OVERFLOW`

Test case *2*: Initially the bucket had *10* litres of water, we then added *10* litres of water for *6* hours. Thus, the total bucket inflow was $10 + 10 \times 6 = 70$ litres. Since this is equal to the capacity of *70* litres, the bucket will be `FILLED`

Test case *3*: Initially the bucket had *2* litres of water, we then added *4* litres of water for *3* hours. Thus, the total bucket inflow was $2 + 4 \times 3 = 14$ litres. Since this is lesser than the capacity of *100* litres, the bucket will be `UNFILLED`.

Test case *4*: Initially the bucket had *3* litres of water, we then added *2* litres of water for *1* hours. Thus, the total bucket inflow was $3 + 2 \times 1 = 5$ litres. Since this is more than the capacity of *3* litres, the bucket will `OVERFLOW`.

Problem Name: Practice - Music Player

Problem Statement:

In this problem, you need to implement the functionality of a music player using doubly linked list!

The functionality of a playlist queue needs to be implemented, i.e., adding a song to the queue, playing the next song, playing the previous song, switching to a song, etc.

You need to complete the following functions:

1. addSong(int songId): Add the songId to the end of the list
2. playNext(): Go to the next song in the list
3. playPrev(): Go to the previous song in the list
4. switchSong(int songId): Find in the list where this songId is present, go to that song and continue the playlist from there (In playlist 1<->2<->3<->4<->5 if the current song is 4 and function is called for 2, the current song is stopped and now order is 2->3->4->5)
5. current(): Return the songId of the song currently playing

Assumptions:

1. playNext function will not be called from the last song
2. playPrev function will not be called from the first song
3. The songId provided in switchSong function is always present in the list
4. The addSong function will not be called for a songId which is already present in the list
5. The current function will not be called when the list is empty.

Note: Until the next function is called, the current song will automatically be the first songId added in the list.

The problem input is query-based:

1 : An integer `songId` will be provided with which addSong(songId) will be called
2 : playNext() is called
3 : playPrev() is called
4 : An integer `songId` will be provided with which switchSong(songId) will be called
5 : current() is called and you need to return the songId for the current song playing

You only need to make changes in the functions mentioned. The input and calling the required functions are taken care of. Do not output anything or you may get WA verdict

Input Format

- The first line of input will contain a single integer *N*, denoting the number of queries.
- The next *N* lines contain a single or two integers depending on the type of query.

Output Format

Output a single integer songId on a new line for every time query type is 5.

Constraints

- $1 \leq N \leq 100000$

- $1 \leq q\ songId \leq q\ 10^9$

Sample Input 1:

16 1 1 1 2 1 3 5 2 5 3 5 1 4 1 5 2 2 4 1 5 2 5

Sample Output 1:

1 2 1 1 2

Explanation:

The current playlist is empty.
1 1 : Add song 1 to the end of list and the current song is now 1.
Playlist: 1
1 2 : Add song 2 to the end of list.
Playlist: 1<->2
1 3 : Add song 3 to the end of list.
Playlist: 1<->2<->3
5 : Output the current song.
2 : The current song is now 2.
5 : Output the current song.
3 : The current song is now 1.
5 : Output the current song.
1 4 : Add song 4 to the end of list.
Playlist: 1<->2<->3<->4
1 5 : Add song 5 to the end of list.
Playlist: 1<->2<->3<->4<->5
2 : The current song is now 2.
2 : The current song is now 3.
4 1 : Switch from 3 to 1. The current song is now 1.
5 : Output 1.
2 : The current song is now 2.
5 : Output 2.

Problem Name: Chef Solves LRU Cache

Problem Statement:

Chef wants to build a data structure that behaves like a **Least Recently Used (LRU) cache** to efficiently manage key–value pairs.

Chef has a **fixed cache capacity** and wants to store keys with their values such that **when the cache exceeds this capacity, the least recently used key is removed**. Chef also wants to retrieve values by keys and update or add new key–value pairs efficiently.

Both retrieval and insertion operations must run in **average** $O(1)$ **time**.

Class Description

Chef will implement a class named *LruCache*.

The class supports the following operations:

- *getValue(cacheKey)* Returns the value associated with *cacheKey* if it exists in the cache; otherwise returns *–1*.

- *putValue(cacheKey, cacheValue)* Inserts or updates the key–value pair in the cache.
  If the cache is already full, the **least recently used key** is evicted before inserting the new pair.

Constraints

- $1 \le T \le 10^5$
- $1 \le cacheCapacity \le 3000$
- $1 \le Q \le 2 \times 10^5$
- $0 \le cacheKey \le 10^4$
- $0 \le cacheValue \le 10^5$
- The total number of *getValue* and *putValue* operations across all test cases does not exceed 2×10^5.

Input Format

- The first line contains a single integer T — the number of test cases.
- For each test case:
  - The first line contains an integer *cacheCapacity* — the capacity of Chef's LRU cache.
  - The next line contains an integer Q — the number of operations.

- Each of the next *Q* lines contains one of the following operations:
- *get* \ *cacheKey* — retrieve the value associated with *cacheKey*.
- *put* \ *cacheKey* \ *cacheValue* — insert or update the key–value pair.

Output Format

- For each *get* operation, print a single line containing:
  - the value associated with *cacheKey* if it exists, or
  - *–1* if the key does not exist in the cache.
- print None for *put* operation.

Sample Input 1:

1 2 9 put 1 1 put 2 2 get 1 put 3 3 get 2 put 4 4 get 1 get 3 get 4

Sample Output 1:

None None 1 None -1 None -1 3 4

Explanation:

- **put(1, 1)**
  Cache: `{1=1}`
  Output: `None`

- **put(2, 2)**
  Cache: `{1=1, 2=2}`
  Output: `None`

- **get(1)**
  Returns 1 and marks key 1 as most recently used
  Cache order: `{2=2, 1=1}`
  Output: 1

- **put(3, 3)**
  Cache is full ? least recently used key is 2, so it is evicted
  Cache: `{1=1, 3=3}`
  Output: `None`

- **get(2)**
  Key 2 is not present in the cache
  Output: `-1`

- **put(4, 4)**
  Cache is full ? least recently used key is 1, so it is evicted
  Cache: `{3=3, 4=4}`
  Output: `None`

- **get(1)**
  Key 1 is not present in the cache
  Output: `-1`

- **get(3)**
  Returns 3 and marks key 3 as most recently used
  Cache order: `{4=4, 3=3}`
  Output: 3

- **get(4)**
  Returns 4
  Output: 4